Text Detection for Manga: CS435 Project Report

1. Introduction

The increasing popularity of manga has created a high demand for rapid, quality translation; however, the manual process of typesetting and translation remains labor-intensive. This project aims to develop an automated pipeline for manga translation, specifically focusing on the first step: the accurate detection and segmentation of speech bubbles.

Our work explores the transition from classical computer vision techniques to modern machine learning. We initially attempted a classical approach using OpenCV for preprocessing and contour detection, but found these methods struggled with the high variability of manga pages. Consequently, we pivoted to a deep learning approach, implementing a U-Net architecture with a VGG-16 encoder for semantic segmentation.

By training on benchmarks like Manga109 and the MangaSeg dataset, we achieved a high level of precision in generating pixel-level segmentation masks. This report details our methodology, the challenges encountered with classical segmentation, and a comparison of models trained on different datasets.

2. Problem

There is a massive amount of web comics and manga (Japanese comic books) published that never receive an official translation. Fan translators may spend a significant amount of time manually cleaning raw pages, which involves erasing the original Japanese text from speech bubbles before they can typeset the translation. Alternatively, readers wanting to either (1) look up a word or (2) get a translation would enjoy a tool that can automatically detect text on the pages.

Our project aims to build an automated image processing pipeline that detects and isolates speech bubbles in raw comic pages. By reliably extracting these text regions and masking the bubbles, we can create a clean background for fan translators or feed the isolated text blocks directly into an OCR and translation API for automated reading.

3. Initial Plan

Our initial plan was to divide the project into different phases. Our first phase was going to be image preprocessing through OpenCV by converting the image to grayscale, blurring it with a Gaussian kernel to remove the screentone noise, and then using adaptive thresholding in order to binarize it.

Our next step was to detect the speech bubbles through OpenCV as well. We did so by using contour detection to find closed white shapes which could have been potential speech bubbles, and filtering them by size and aspect ratio.

We then planned to extract the speech bubbles so that they can be translated. We did so by using EasyOCR which uses a special neural network to identify Japanese characters.

Lastly, we also wanted to use drawing functions in OpenCV to paint over the Japanese characters with white boxes that were filled with English text.

4. Attempts at Classical CV

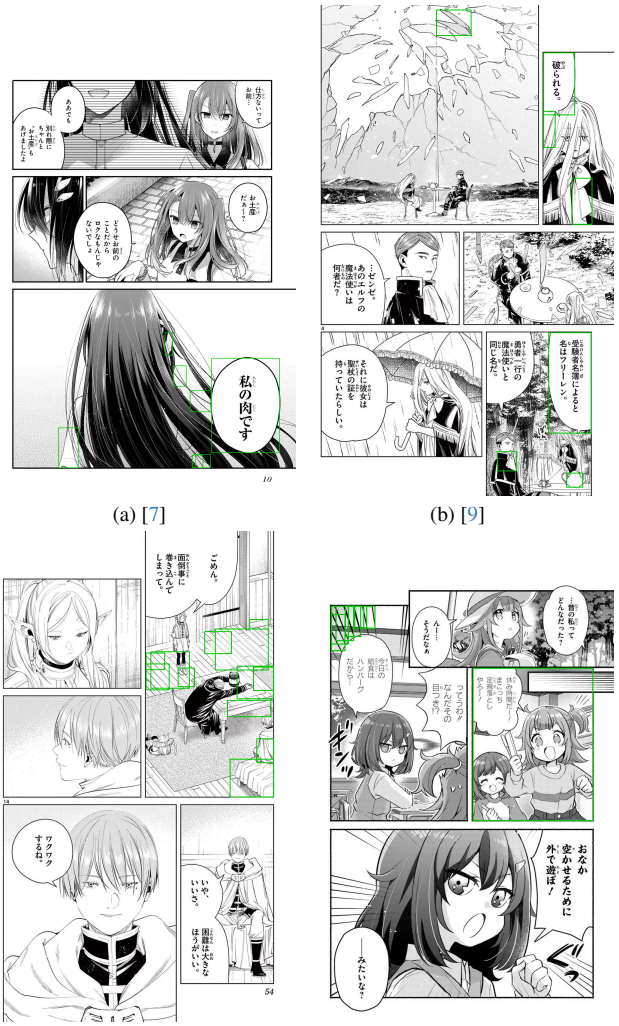
Our attempts at classical CV using OpenCV did not go well for numerous reasons. Our preprocessing pipeline converted the image to grayscale, applied a Gaussian blur to reduce screentone noise, and used adaptive thresholding to binarize the image. We then applied contour detection to find closed white shapes and filtered them by size and aspect ratio as potential speech bubble candidates.

See Figure 1 for example 'failed' attempts.

The most immediate issue was screentone noise. Manga pages frequently use halftone dot patterns to simulate shading, and Gaussian blur is not effective enough to suppress this. After thresholding, these patterns produced large connected white regions that were incorrectly detected as speech bubbles. Increasing the blur reduced this effect but also destroyed the fine edges of actual speech bubbles, making contour detection less reliable.

Speech bubbles come in an enormous variety of shapes: circular, rectangular, jagged, cloud-like, or none at all. Our size and aspect ratio filters could handle simple cases but broke down on anything unusual. Overlapping panels, bubbles that bled across panel borders, and text placed directly on dark backgrounds all produced either missed detections

or large numbers of false positives with no clear systematic way to distinguish them.



(a) [7] (b) [9]

(c) [9] (d) [2]

Figure 1. Example output attempts at classical CV. Green bounding boxes denote detected speech bubbles. We observe little success in accuracy, alongside high amounts of false-positives.

Ultimately, every parameter we changed to fix one type of failure made another worse. There was no single threshold, kernel size, or filter combination that resulted in a generic success across the variety of manga pages we tested on.

5. Pivot to ML-based Approach

5.1. Overview

We came to no decent results after a few days of attempting to implement classical CV. After some discussion, we came to the conclusion that manga pages are *too variable* for any single classical CV algorithm to be highly accurate in its segmentation. This is corroborated by a 2020 study, which came to a similar conclusion [4].

Thus, we decided to scrap our initial plan outlined in Section 3 and instead opt for an approach based in machine learning. Looking around at various online resources and existing implementations, we came across a study that implements speech bubble detection in comic books using a deep CNN [3]. We decided to re-implement this architecture as the basis of our approach.

The task is framed as semantic segmentation: given a raw manga page, predict a pixel-level mask of speech bubble regions. We use a U-Net with a VGG-16 encoder pretrained on ImageNet, following Dubray & Laubrock. Prior work has shown that features learned on natural photographs transfer surprisingly well to illustrated images [3]. The decoder is trained from scratch to upsample these features back to full resolution, producing the final segmentation mask. We train using Dice loss, which optimizes the overlap between prediction and ground truth masks. Del Gobbo (2020) demonstrates that Dice loss outperforms cross-entropy for this task [4]. We then write a Python pipeline in order to prepare the wanted data to pass into the model for training. All training was conducted on an NVIDIA T4 GPU via Google Colab.

5.2. Training and Datasets

We trained and evaluated the model in two phases. In the first phase, we used the open-mantra-dataset, a small publicly available manga dataset of approximately 168 annotated pages [5]. As only bounding box annotations were available, we rasterized these into filled rectangle masks as a ground truth approximation. Despite the coarse supervision, the model achieved an F1 score of 0.927 on the validation set, confirming that the architecture and training pipeline were functioning correctly.

In the second phase, we switched to Manga109, the standard benchmark dataset for manga analysis, paired with MangaSeg, a CVPR 2025 dataset providing pixel-level segmentation masks for the full Manga109 dataset [1] [8]. This gives us approximately 9,900 annotated pages with precise pixel-level contours, a substantial improvement over the rectangle approximations used in phase one. Training on this data brought the model to an F1 score of 0.975 on the validation set.

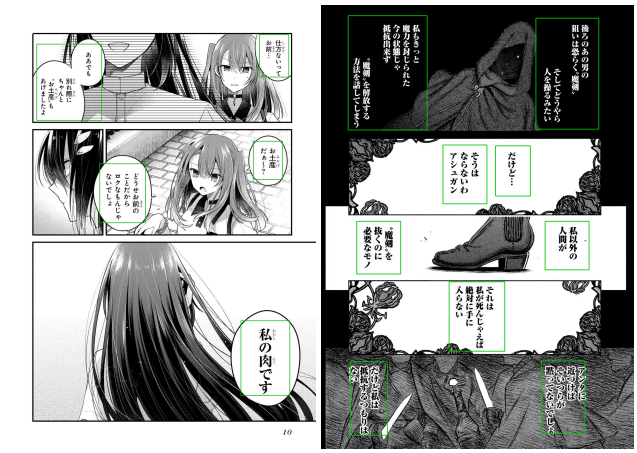
6. Example Output & Translation Pipeline

6.1. open-mantra-dataset-trained Model

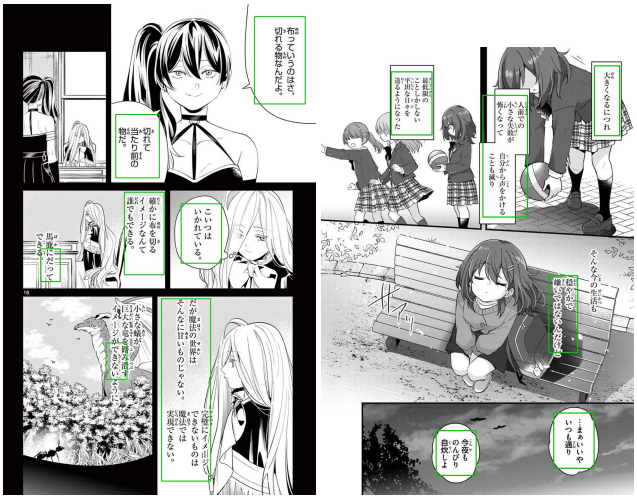
See Figure 2 below.

6.2. Manga109+MangaSeg-trained Model

See Figure 3 below.

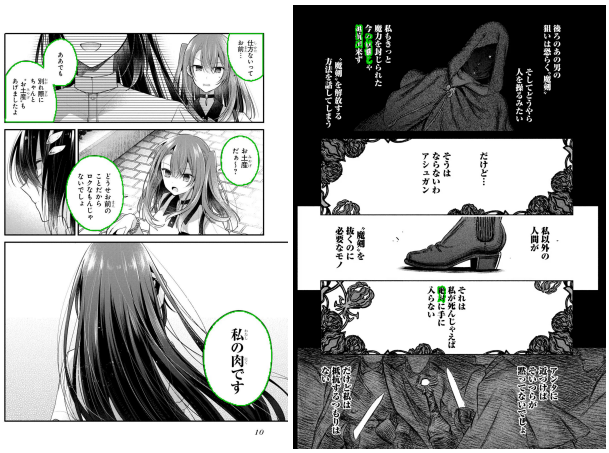


(a) [7] (b) [6]

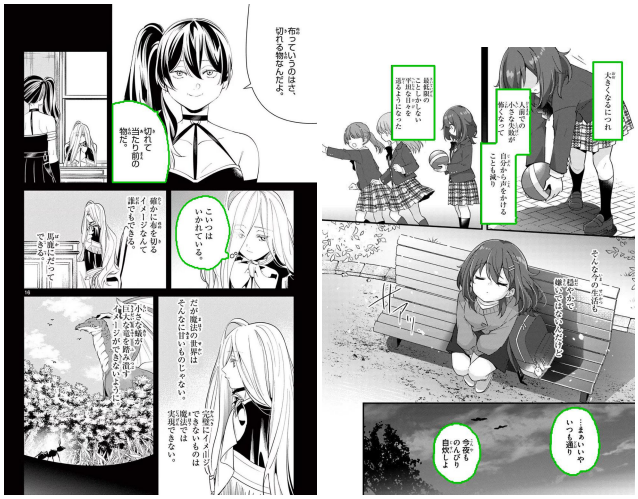


(c) [9] (d) [2]

Figure 2. Example output from our open-mantra-dataset trained model. We observe bounding boxes that denote accurate speech bubble text detection, as well as decent floating text detection.



(a) [7] (b) [6]



(c) [9] (d) [2]

Figure 3. Example output from our Manga109+MangaSeg trained model. We observe pixel-level contours that denote highly accurate speech bubble detection, but poor floating text detection.

6.3. Auto-translation Pipeline

The segmentation model serves as the first stage of a fully automated translation pipeline. The detected regions are cropped from the original image using the predicted contours, filtered to remove noise below 1,000 pixels, and passed through the following steps:

1. Get segmentation mask of image from trained model
2. Crop detected text regions
3. Pass cropped images through Japanese OCR
4. Pass raw text from OCR to Google Translate API
5. Overlay translated text onto image using OpenCV

An example output is shown in Figure 4.

7. Discussion & Conclusion

7.1. Model Comparison

The two models we trained have their own strengths and weaknesses. The open-mantra-dataset-trained model identifies floating text more often, but only gives loose bounding boxes for each text section detected. On the other hand, the Manga109+MangaSeg-trained model gives an accurate, pixel-level segmentation mask, but almost never identifies floating text.

Depending on the use case, both models could be helpful. For example, if we wanted to just extract the text, the former would be better, while if we wanted to create an overlay on the actual image, the latter would be better. There is also the idea that we could also use *both* models in order to cover each other's faults (one to extract the text,



Figure 4. Example auto-translated image [2]

and one to create an accurate mask over the bubbles!).

We believe that the Manga109+MangaSeg dataset definitely has more potential than the model we arrived at. While the open-mantra-dataset was small with comprehensive text annotations, the Manga109+MangaSeg dataset was very big, with the "speech bubble" subset of annotations being much larger than the "floating text" subset.

If we were to retrain it, we would probably want to implement some sort of weighted dice loss in order to put more emphasis on training the "floating text" subset (the dataset splits the segmentation masks into different categories, "floating text" being one of them). This would have been our intention had we not run into limitations with Google Colab, along with the fact that training this with dataset takes many hours.

7.2. Translation Pipeline

Our sample output shows a working automatic translation pipeline. While the output can obviously be improved visually (by, for example, writing a better overlay script, or using something better than Google Translate's API), we think of it as more of a proof of concept. It shows that once you get a working model, there are many potential ways one can use the text. Some examples include (1) using a better translation API, (2) passing the text from OCR to a human translator, (3) using the raw Japanese text as a tool for language learning, etc.

7.3. Reflection

This project reinforced what we learned in class in regards to computer vision: classical methods that work well on constrained, structured inputs often break when applied to something as variable as manga. The failure of our classical CV pipeline attempt speaks to a mismatch between our assumptions of rule-based algorithms and the reality of the data. In contrast, the success of our deep learning approach required no specific rules at all as the model learned what a "speech bubble" and "manga text" looks like entirely from labeled examples.

7.4. Conclusion

Our project demonstrates that classical computer vision is insufficient for reliable manga speech bubble detection, and that a deep learning approach based on semantic segmentation is both feasible and effective. We showed that the segmentation model we trained can be integrated into a fully automated translation pipeline, from raw manga page to English text overlay. While the output is more of a proof of concept than a polished product, it shows that the core problem of speech bubble detection is largely solved, and that the remaining challenges, such as floating text, OCR accuracy, and translation quality, could be explored in the future.

References

[1] Kiyoharu Aizawa, Yusuke Matsui, JeongHun Baek, Hikaru Ikuta, and Yingxuan Li. Manga109 dataset and creation of metadata. *Information Technology Center, The University of Tokyo*, 2024. 2

[2] Atto and Quro. *Hibi wa Sugiredo Meshi Umashi*. Kadokawa, 2025. 2, 3, 4

[3] David Dubray and Jochen Laubrock. Deep cnn-based speech balloon detection and segmentation for comic books, 2019. 2

[4] Julián Del Gobbo and Rosana Matuk Herrera. Unconstrained text detection in manga: a new dataset and baseline, 2020. 2

[5] Ryota Hinami, Shonosuke Ishiwatari, Kazuhiko Yasuda, and Yusuke Matsui. Towards fully automated manga translation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021. OpenMantra Dataset: https://github.com/mantra-inc/open-mantra-dataset. 2

[6] Kousuke Satake. *Majo to Yajuu*. Kodansha, 2016. 3

[7] Naoki Shigeno. *Watashi o Taberu, Hitodenashi*. Kadokawa, 2020. 2, 3

[8] Minshan Xie, Jian Lin, Hanyuan Liu, Chengze Li, and Tien-Tsin Wong. Advancing manga analysis: Comprehensive segmentation annotations for the manga109 dataset. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 2

[9] Kanehito Yamada and Tsukasa Abe. *Sousou no Frieren*. Shogakukan, 2020. 2, 3